

```
1 # 1. Upload the kaggle.json file
2 from google.colab import files
3 files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username": "jeniljethva", "key": "3d322e27f15c47bbb127a7e067918580"}'}

```
1 # 2. Move it to the correct folder
2 !mkdir -p ~/.kaggle
3 !cp kaggle.json ~/.kaggle/
4 !chmod 600 ~/.kaggle/kaggle.json
```

```
1 # 3. Download your face mask dataset
2 !kaggle datasets download -d andrewmvd/face-mask-detection
```

Dataset URL: <https://www.kaggle.com/datasets/andrewmvd/face-mask-detection>
License(s): CC0-1.0
Downloading face-mask-detection.zip to /content
100% 398M/398M [00:24<00:00, 17.3MB/s]

```
1 # 4. Unzip the images and XML files
2 !unzip face-mask-detection.zip -d mask_data
```

Show hidden output

STEP-1

```
1 # Breaking Down the XML Tags
2 # Think of the XML as a "map" for its corresponding PNG image (maksssksksss0.png).
3
4 # <size>: This tells the model the dimensions of the original image (Width: 512px, Height: 366px). The depth: 3 means it's a color image (RGB).
5
6 # <object>: Each person's face in the image is an "object." This file describes three different faces found in one image.
7
8 # <name>: This is your label (e.g., without_mask or with_mask).
9
10 # <bndbox> (Bounding Box): This is the most important part. It provides the $(x, y)$ coordinates for a rectangle drawn around the face:
11
12 # xmin, ymin: The top-left corner of the face.
13
14 # xmax, ymax: The bottom-right corner of the face
```

```
1 # step -1 Open the PNG image.
2
3 # step -2 Open the matching XML file.
4
5 # step -3 Use the bndbox coordinates to "crop" the faces out of the big image or draw boxes around them during training
6 # so the model learns: "This specific group of pixels equals a mask."
```

```
1 # Since PyTorch, cannot feed the XML text directly into a Neural Network. we have to "parse" it.
2
3 # Loops through every XML file.
4
5 # Extracts the xmin, ymin, xmax, ymax and the name.
6
7 # Crops the face from the PNG image using those coordinates.
8
9 # Saves that small face image into a list (like the "data" list in your YouTube video) to be used for training.
```

STEP-2

```
1 # Step 2: XML Parsing (The "Map" Reader)
2 # we can't feed XML to a neural network. we need a function that reads the XML and extracts:
3
4 # The Label (with_mask, without_mask).
5
6 # The Coordinates (xmin, ymin, xmax, ymax)
```

STEP-3

```
1 # Step 3: Dataset Preparation (Face Cropping)
2 #the images are not "just faces." In this Kaggle dataset, one image might have 10 people. we need to "crop" each face out using
3 # the coordinates from Step 2.
4
5 # The Logic:
6
7 # Open the PNG image.
8
9 # For every <object> in the XML, crop that specific rectangle.
10
11 # Resize that crop to a standard size (e.g., 128x128) so the CNN can process it.
```

Implementation

```
1 # import os
2 # import xml.etree.ElementTree as ET
3 # from PIL import Image
4 # import torch
5 # from torch.utils.data import Dataset
6 # from torchvision import transforms
7 import os
8 import xml.etree.ElementTree as ET
9 from PIL import Image
```

```
1 # 1. import os
2
3 # What it is: Python's built-in Operating System module.
4 # What it does in your project: It allows your Python code to interact with your computer's (or Google Colab's) file system and folders.
5 # Where you used it in your code:
6 # Joining folder paths safely: os.path.join(image_dir, img_file)
7 # Listing all files in a folder: os.listdir(annot_dir)
```

```

8 # Checking if an image exists: os.path.exists(img_path)
9
10 # 2. import xml.etree.ElementTree as ET
11
12 # What it is: Python's built-in tool for parsing and reading XML data.
13 # The as ET part gives it a short nickname (ET) so you don't have to type the long name every time.
14 # What it does in your project: XML files are structured like a tree (with roots, branches, and tags like <object> and <bndbox>).
15 # This library navigates through the XML file to read the face labels and coordinates.
16 # Where you used it in your code:
17 # Reading the XML file: tree = ET.parse(xml_file)
18 # Finding the face tags: root.findall('object')
19
20 # 3. from PIL import Image
21
22 # What it is: Pillow (Python Imaging Library), the standard Python library for working with images.
23 # What it does in your project: It allows Python to open, manipulate, crop, display, and save actual image files (like .png or .jpg).
24 # Where you used it in your code:
25 # Opening the image: Image.open(img_path).convert("RGB")
26 # Cropping the face using coordinates: image.crop((box[0], box[1], box[2], box[3]))

```

Double-click (or enter) to edit

```

1 # # 1. Define the Label Map
2 # # We convert text labels to numbers
3 #
4 # 1. Update your label map to ONLY contain 2 classes
5 label_map = {
6     "without_mask": 0,
7     "with_mask": 1
8 }
9
10 def parse_xml_binary(xml_file):
11     """Reads an XML file and returns bounding boxes and labels for binary classes."""
12     tree = ET.parse(xml_file)
13     root = tree.getroot()
14     boxes = []#Will hold lists of coordinates like [, [79, 105, 109, 142], ].
15     labels = []#Will hold corresponding numeric class labels like [1].
16
17     for obj in root.findall('object'):#Finds every <object> tag in the XML
18         label_text = obj.find('name').text
19
20         # CRITICAL CHANGE: If the label is incorrect wear, skip it entirely!
21         if label_text == "mask_wearred_incorrect":
22             continue
23
24         if label_text in label_map:
25             labels.append(label_map[label_text])
26             bndbox = obj.find('bndbox')
27             xmin = int(bndbox.find('xmin').text)
28             ymin = int(bndbox.find('ymin').text)
29             xmax = int(bndbox.find('xmax').text)
30             ymax = int(bndbox.find('ymax').text)
31             boxes.append([xmin, ymin, xmax, ymax])
32
33     return boxes, labels
34

```

```

35 def prepare_data_binary(image_dir, annot_dir):
36     """Crops faces belonging ONLY to class 0 and class 1."""
37     cropped_faces = []
38     face_labels = []
39     annots = sorted(os.listdir(annot_dir))
40     #["maksssksksss0.xml", "maksssksksss1.xml", ...]
41
42     for xml_file in annots:
43         img_file = xml_file.replace('.xml', '.png')
44         img_path = os.path.join(image_dir, img_file)#image_dir ("/content/mask_data/images") + img_file ("maksssksksss0.png") ->"/content/mask_data/images/maksssksksss0.png"
45         xml_path = os.path.join(annot_dir, xml_file)#annot_dir ("/content/mask_data/annotations") + xml_file ("maksssksksss0.xml") -> "/content/mask_data/annotations/makss
46
47         if not os.path.exists(img_path):
48             continue
49
50         # Check if this XML contains valid binary targets before opening the image
51         boxes, labels = parse_xml_binary(xml_path)
52         if not boxes:
53             continue
54
55         image = Image.open(img_path).convert("RGB")
56         for box, label in zip(boxes, labels):
57             face = image.crop((box[0], box[1], box[2], box[3]))
58             cropped_faces.append(face)
59             face_labels.append(label)
60
61     return cropped_faces, face_labels
62 #     cropped_faces = [
63 #         <PIL.PngImagePlugin.PngImageFile image mode=RGB size=30x37 at 0x7F8B2C10A110>,
64 #         <PIL.PngImagePlugin.PngImageFile image mode=RGB size=42x51 at 0x7F8B2C10A290>,
65 #         <PIL.PngImagePlugin.PngImageFile image mode=RGB size=35x40 at 0x7F8B2C10A350>
66 #     ]
67 # face_labels = [1, 0, 1, 1, 0, 0, 1]
68
69 # Colab Paths
70 image_path = "/content/mask_data/images"
71 annot_path = "/content/mask_data/annotations"
72
73 faces, labels = prepare_data_binary(image_path, annot_path)
74 print(f"Success! Total faces extracted: {len(faces)}")
75

```

Success! Total faces extracted: 3949

```

1 from collections import Counter
2
3 # Reverse the label map to look up names by their integer values
4 inverse_label_map = {v: k for k, v in label_map.items()}
5
6 # Count the occurrences of each label index
7 label_counts = Counter(labels)
8
9 print("--- Dataset Class Distribution ---")
10 for label_idx, count in label_counts.items():
11     class_name = inverse_label_map.get(label_idx, "Unknown")
12     print(f"{class_name:<25} (Label {label_idx}): {count} faces")

```

```
--- Dataset Class Distribution ---
without_mask      (Label 0): 717 faces
with_mask         (Label 1): 3232 faces
```

```
1 # successfully turned 853 crowd images into 4,072 individual face images.
```

- **conclusion**
- Now that we have a list of faces (PIL images) and labels (0, 1, or 2), we need to wrap them into a PyTorch Dataset.
- **STEP-4**
- Building the PyTorch Dataset Class In PyTorch, we wrap all the logic List item above into a Dataset class.

```
1 # This class tells PyTorch: __len__: How many total faces do we have? __getitem__: Give me the $i$-th face and its label.
```

STEP 5: Data Augmentation & Normalization

```
1 # To make your model "smart," you should use torchvision.transforms.
2
3 # Resize: All faces must be the same size.
4
5 # ToTensor: Convert pixels (0-255) to Tensors (0.0-1.0).
6
7 # Normalize: Adjust the mean and standard deviation so the model learns faster.
```

```
1 # Step 4 & 5: Building the PyTorch Dataset and DataLoaders
```

```
1 # . The MaskDataset Class (The Factory Worker)
2 # PyTorch is a very structured framework. It doesn't like dealing with raw Python lists directly during training. It requires a special container called a Dataset.
3
4 # Think of the MaskDataset class as a worker in a factory warehouse.
5 # You hand this worker all your cropped face images and labels. It provides three specific rules for how to handle them:
6
7 # __init__: The worker takes your raw lists (faces, labels) and your transformation rules, storing them safely in the warehouse.
8
9 # __len__: If the AI model asks, "How many total faces do we have?", this function quickly counts them and answers.
10
11 # __getitem__: This is the most important one. When the AI says, "Give me face number 42," this function grabs face 42,
12 # applies your image transformations (resizing, converting to math tensors), and hands over the finished image and its label.
```

```
1 from torch.utils.data import Dataset, DataLoader, random_split
2 from torchvision import transforms
3
4 class MaskDataset(Dataset):
5     def __init__(self, faces, labels, transform=None):
6         self.faces = faces
7         self.labels = labels
8         self.transform = transform
9
10    def __len__(self):
11        return len(self.faces)
```

```

12
13     def __getitem__(self, idx):
14         face = self.faces[idx]
15         label = self.labels[idx]
16
17         if self.transform:
18             face = self.transform(face)
19
20         return face, label

```

```

1 # Step 4: The PyTorch Dataset Class In PyTorch, the Dataset class acts as the bridge between
2 # your raw data (those 4,072 faces) and the model.
3 # We also apply Transforms here to make sure every face is resized to exactly the same dimensions (like 128 \times 128)
4 # and converted into a math-friendly "Tensor."

```

```

1 # Step 5: Define Transforms
2 # We resize to 128x128 and Normalize using standard ImageNet values
3 # to help the model learn faster.

```

```

1 from sklearn.model_selection import train_test_split
2 from torch.utils.data import DataLoader
3 import torch
4
5 # 1. Stratified Split of your raw list of faces and labels (Automatically handles 2 classes now!)
6 train_faces, val_faces, train_labels, val_labels = train_test_split(
7     faces, labels, test_size=0.2, stratify=labels, random_state=42
8 )
9 #stratify=labels Remember your class distribution? You have roughly 4x more "with mask" images than "without mask" images.
10 #If you split the data randomly, you might accidentally put all the "without mask" images into the training set,
11 #leaving none for the test set. stratify forces the split to maintain that exact 1:4 ratio in both the training and validation sets.
12
13 # 2. Separate transform pipelines
14 train_transforms = transforms.Compose([
15     transforms.Resize((128, 128)),
16     transforms.ToTensor(),
17     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
18 ])
19
20 val_transforms = transforms.Compose([
21     transforms.Resize((128, 128)),
22     transforms.ToTensor(),#It converts the image from an RGB image into numbers between 0.0 and 1.0.
23     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])#It adjusts the brightness and contrast values using standard numbers (mean and std)
24     #calculated from millions of general images. This helps the model learn much faster because the data behaves predictably.
25 ])
26
27 # 3. Wrap into your MaskDataset class separately
28 train_data = MaskDataset(train_faces, train_labels, transform=train_transforms)
29 val_data = MaskDataset(val_faces, val_labels, transform=val_transforms)
30
31 # 4. Create DataLoaders
32 train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
33 val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
34
35 print(f"Training samples: {len(train_data)}")
36 print(f"Validation samples: {len(val_data)}")

```

```

37
38 # 5. Compute new binary weights to handle remaining imbalance between class 0 and 1
39 total_train = len(train_labels)
40 class_counts = [train_labels.count(0), train_labels.count(1)]
41
42 # Formula: Total Samples / (Number of Classes * Class Count)
43 weight_0 = total_train / (2.0 * class_counts[0]) # Without Mask Penalty    2.75
44 weight_1 = total_train / (2.0 * class_counts[1]) # With Mask Penalty      0.61
45
46 # Safely create the device variable if it hasn't been declared yet
47 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
48
49 # Convert new 2-class weights into a tensor and move to GPU/CPU
50 class_weights = torch.tensor([weight_0, weight_1], dtype=torch.float).to(device)
51
52 print(f"New 2-Class Weights -> Without Mask (Label 0): {weight_0:.2f} | With Mask (Label 1): {weight_1:.2f}")

```

Training samples: 3159
 Validation samples: 790
 New 2-Class Weights -> Without Mask (Label 0): 2.75 | With Mask (Label 1): 0.61

```

1 # What's happening here?
2 # __getitem__: This is like a waiter. When the model says "I'm ready for face #50,"
3 # this function grabs the face, resizes it, turns it into a tensor, and hands it over with the label.
4
5 # random_split: We never train on our whole dataset.
6 # We keep 20% aside (the "Validation set") to test the model on faces it has never seen before.
7
8 # DataLoader: This feeds the model in "batches" of 32 faces at a time.
9 # This is much more memory-efficient than trying to load all 4,072 faces at once!

```

```

1 # Checkpoint: Let's see a face!
2 # Before we build the CNN,
3 # it's good practice to visualize one "batch" to make sure the images look right after all those transforms. Run this:

```

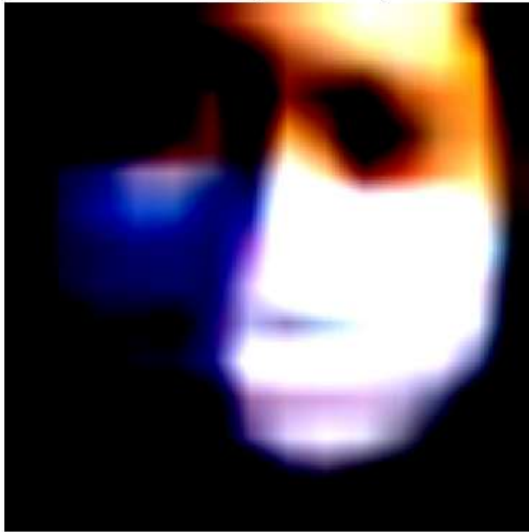
```

1 import matplotlib.pyplot as plt
2
3 # 1. Get one batch of images from your new 2-class train_loader
4 images, batch_labels = next(iter(train_loader))
5
6 # 2. Map the binary numbers to human-readable names
7 binary_classes = {0: "Without Mask", 1: "With Mask"}
8 current_label_num = batch_labels[0].item()
9 current_label_text = binary_classes[current_label_num]
10
11 # 3. Show the first image in the batch
12 # FIX: Permute must change (C, H, W) -> (H, W, C) by using (1, 2, 0)
13 plt.imshow(images[0].permute(1, 2, 0))
14 plt.title(f"Label: {current_label_num} ({current_label_text})")
15 plt.axis('off') # Hides the pixel coordinate grid lines for a cleaner look
16 plt.show()

```

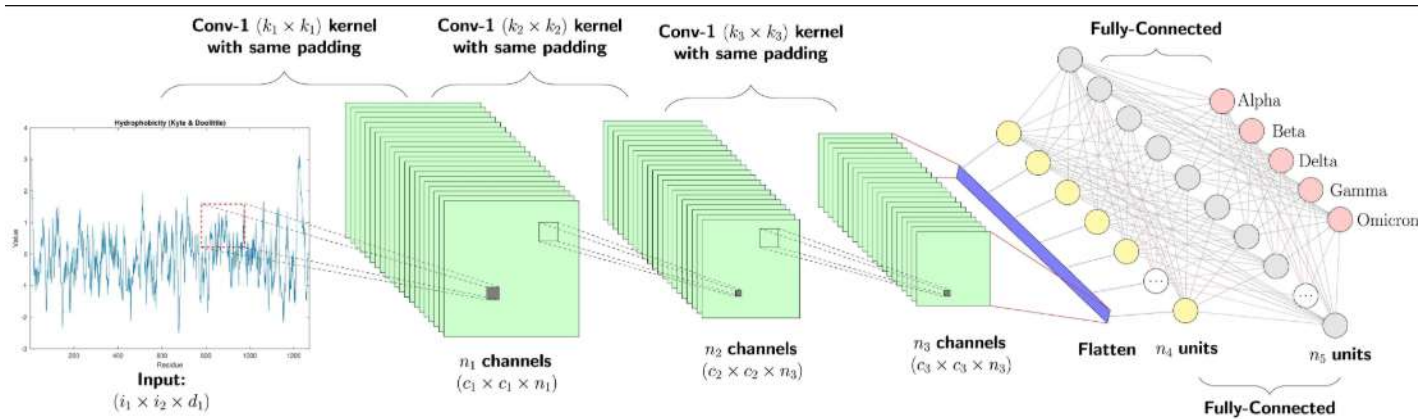
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.1007793..2.169412].

Label: 1 (With Mask)



STEP - 6: Defining the CNN Architecture

```
1 # In PyTorch, we define our model as a class. We'll use two Convolutional layers, Max-Pooling to shrink the image,  
2 # and then "Linear" (Dense) layers to make the final classification between our 3 classes.
```



```
1 # Input ---> RGB Image --> 3 * 128 * 128  
2 # Conv1 + Pool --> Detect features & Shrink --> 32 * 64 * 64  
3 # Conv2 + PoolDetect ---> complex shapes & Shrink--> 64 * 32 * 32  
4 # FlattenTurn into a list ---> $65,536$ elements  
5 # FC2 (Output)Classify$3$ (Scores for each class)
```

```
1 import torch  
2 import torch.nn as nn
```

```

3 import torch.nn.functional as F
4
5 class FaceMaskCNN(nn.Module):
6     def __init__(self):
7         super(FaceMaskCNN, self).__init__()
8
9         # Layer 1: Input is 3 channels (RGB), Output is 32 features
10        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
11        self.pool = nn.MaxPool2d(2, 2) # Reduces size from 128x128 to 64x64
12
13        # Layer 2: Input is 32 features, Output is 64 features
14        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
15        # Another pool will reduce size from 64x64 to 32x32
16
17        # Fully Connected Layers
18        # After 2 pools, 128x128 becomes 32x32.
19        # So input features = 64 (filters) * 32 * 32 (pixels)
20        self.fc1 = nn.Linear(64 * 32 * 32, 128)
21        self.dropout = nn.Dropout(0.5) # Prevents overfitting
22        self.fc2 = nn.Linear(128, 2) # 3 Classes: Without, With,
23
24    def forward(self, x):
25        # Apply Conv -> ReLU -> Pool
26        x = self.pool(F.relu(self.conv1(x)))
27        x = self.pool(F.relu(self.conv2(x)))
28
29        # Flatten the image into a long vector
30        x = x.view(-1, 64 * 32 * 32)
31
32        # Apply Dense layers
33        x = F.relu(self.fc1(x))
34        x = self.dropout(x)
35        x = self.fc2(x) # Final output (Raw scores/Logits)
36
37        return x
38
39 # Initialize the model and move it to GPU if available
40 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
41 model = FaceMaskCNN().to(device)
42
43 print(model)

```

```

FaceMaskCNN(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=65536, out_features=128, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
  (fc2): Linear(in_features=128, out_features=2, bias=True)
)

```

```

1 # What's happening inside the "Brain"?
2
3 # Conv2d: This scans the image with small filters to find patterns.
4
5 # MaxPool2d: This is like a "summary" layer.
6 # It takes a 2$ \times 2$ square of pixels and only keeps the brightest one, cutting the image size in half.

```

```
7
8 # Flatten (view): Neural networks can't "see" 2D squares at the end.
9 # We have to stretch the pixels out into one long line before feeding them to the final decision-making layers.
10
11 # Dropout: This randomly "turns off" 50% of the neurons during training.
12 # It sounds mean, but it forces the model to not rely on just one feature, making it much smarter and more flexible.
```

```
1 Start coding or generate with AI.
```

STEP 7: Setting the "Rules" (Loss & Optimizer)

```
1 # Before we start the training loop, we need to tell the model how to measure its mistakes (Loss) and how to fix them (Optimizer).
```

```
1 # import torch.optim as optim
2
3 # # CrossEntropyLoss is perfect for multi-class (3 classes)
4 # criterion = nn.CrossEntropyLoss()
5
6 # # Adam is a very popular, "smart" version of Gradient Descent
7 # optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
1 # In Deep Learning, we call one full pass through the entire dataset an Epoch.
2
3 # In each epoch, the model will look at a batch of faces, make a guess, see how far off it was using the Loss Function,
4 # and then the Optimizer will tweak the internal "weights" (the brain's connections) to do better next time.
```

STEP 7: The Training Loop

```
1 import torch.optim as optim
2 import time
3
4 # 1. CRITICAL FIX: Pass the class weights we calculated into CrossEntropyLoss
5 # This tells the model to penalize errors on minority classes heavily!
6 criterion = nn.CrossEntropyLoss(weight=class_weights)
7
8 # Adam optimizer stays the same
9 optimizer = optim.Adam(model.parameters(), lr=0.001)
10
11 epochs = 10
12 train_losses = []
13 val_losses = []
14
15 print(f"Starting training on {device}...")
16
17 for epoch in range(epochs):
18     start_time = time.time()
19
20     # --- TRAINING PHASE ---
21     model.train()
22     running_loss = 0.0
23     correct = 0
24     total = 0
25
```

```

25
26 for images, labels in train_loader:
27     images, labels = images.to(device), labels.to(device)
28
29     # Clear previous gradients
30     optimizer.zero_grad()
31
32     # Forward pass
33     outputs = model(images)
34
35     # Calculate balanced Loss
36     loss = criterion(outputs, labels)
37
38     # Backward pass & Optimize
39     loss.backward()
40     optimizer.step()
41
42     running_loss += loss.item()
43
44     # FIX: Correct equality comparison syntax for training accuracy
45     _, predicted = torch.max(outputs.data, 1)
46     total += labels.size(0)
47     correct += (predicted == labels).sum().item()
48
49 train_loss = running_loss / len(train_loader)
50 train_acc = 100 * correct / total
51 train_losses.append(train_loss)
52
53 # --- VALIDATION PHASE ---
54 model.eval()
55 val_running_loss = 0.0
56 val_correct = 0
57 val_total = 0
58
59 with torch.no_grad():
60     for images, labels in val_loader:
61         images, labels = images.to(device), labels.to(device)
62         outputs = model(images)
63         loss = criterion(outputs, labels)
64         val_running_loss += loss.item()
65
66         # FIX: Correct equality comparison syntax for validation accuracy
67         _, predicted = torch.max(outputs.data, 1)
68         val_total += labels.size(0)
69         val_correct += (predicted == labels).sum().item()
70
71 val_loss = val_running_loss / len(val_loader)
72 val_acc = 100 * val_correct / val_total
73 val_losses.append(val_loss)
74
75 end_time = time.time()
76 print(f"Epoch {epoch+1}/{epochs} | "
77       f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | "
78       f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}% | "
79       f"Time: {end_time - start_time:.2f}s")
80
81 print("Training Complete!")

```

Starting training on cuda...

Epoch 1/10	Train Loss: 0.3515	Train Acc: 87.62%	Val Loss: 0.1159	Val Acc: 96.96%	Time: 9.09s
Epoch 2/10	Train Loss: 0.1288	Train Acc: 95.73%	Val Loss: 0.0751	Val Acc: 96.71%	Time: 3.98s
Epoch 3/10	Train Loss: 0.1153	Train Acc: 96.17%	Val Loss: 0.0830	Val Acc: 95.32%	Time: 4.75s
Epoch 4/10	Train Loss: 0.1101	Train Acc: 96.23%	Val Loss: 0.0695	Val Acc: 97.22%	Time: 4.09s
Epoch 5/10	Train Loss: 0.1055	Train Acc: 96.14%	Val Loss: 0.0648	Val Acc: 97.72%	Time: 3.96s
Epoch 6/10	Train Loss: 0.0887	Train Acc: 96.83%	Val Loss: 0.0732	Val Acc: 96.84%	Time: 5.35s
Epoch 7/10	Train Loss: 0.0843	Train Acc: 97.21%	Val Loss: 0.0978	Val Acc: 96.08%	Time: 4.42s
Epoch 8/10	Train Loss: 0.0957	Train Acc: 96.68%	Val Loss: 0.1206	Val Acc: 96.46%	Time: 4.54s
Epoch 9/10	Train Loss: 0.0941	Train Acc: 96.77%	Val Loss: 0.1148	Val Acc: 95.82%	Time: 4.53s
Epoch 10/10	Train Loss: 0.0770	Train Acc: 97.40%	Val Loss: 0.0681	Val Acc: 97.47%	Time: 4.40s

Training Complete!

1 # Loss: This should decrease every epoch. If it's going down, the model is learning! Validation Accuracy:

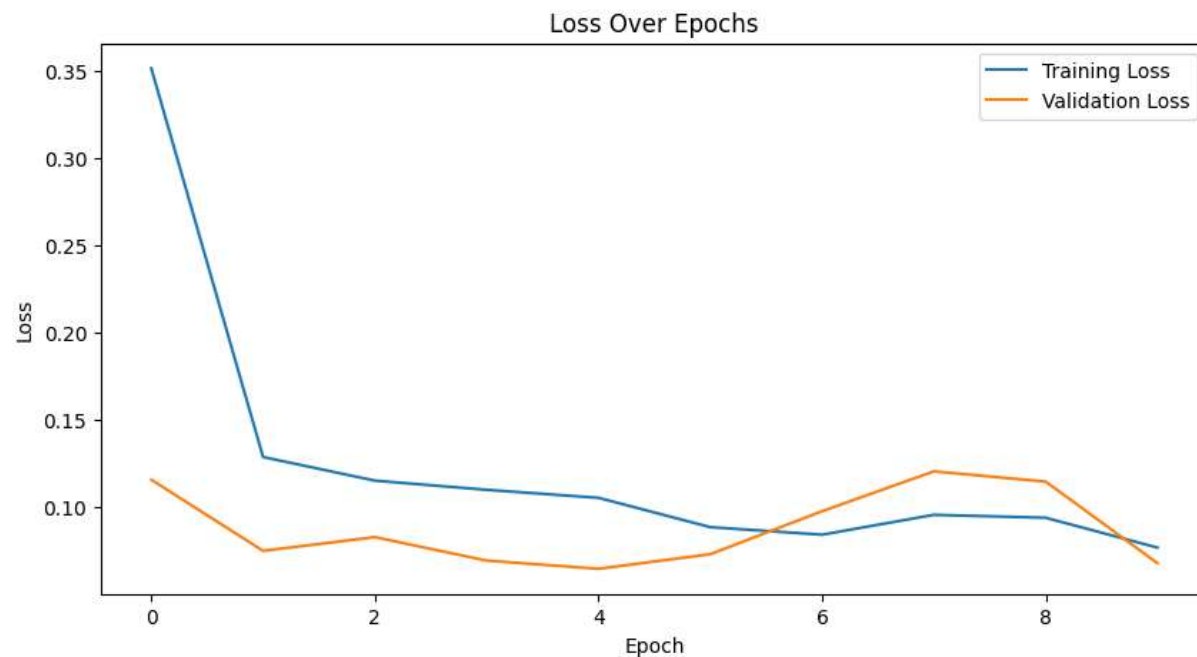
2 # This tells you how well the model performs on "unseen" faces.

3 # If your Training Accuracy is 99% but Validation is only 70%, the model is just memorizing (this is called Overfitting).

STEP-8: Visualizing the Results

1 # If the curves are both going down together, your model is "healthy."

```
1 plt.figure(figsize=(10, 5))
2 plt.plot(train_losses, label='Training Loss')
3 plt.plot(val_losses, label='Validation Loss')
4 plt.title('Loss Over Epochs')
5 plt.xlabel('Epoch')
6 plt.ylabel('Loss')
7 plt.legend()
8 plt.show()
```



```

1 # Analyzing Your Graph
2 # Looking at your plot, here is what your "brain" is telling you:
3
4 # Convergence: Both curves drop sharply at the start, which means your model learned the basic features (like the shape of a mask) very quickly.
5
6 # A "Healthy" Gap: The training loss (blue) is slightly lower than the validation loss (orange) at the end.
7 # This is perfect. It means your model is generalizing well.
8
9 # The Little Bump at Epoch 10: Notice how the orange line (Validation Loss) ticks upward slightly at the very end?
10 # This is a tiny hint of Overfitting.
11 # It means the model is starting to memorize the specific pixels in your training set rather than learning general features.
12 # You stopped at the perfect time!

```

STEP 9: Testing on a Real Image

```

1 from PIL import Image
2 import matplotlib.pyplot as plt
3 from torchvision import transforms
4 import torch
5
6 def predict_mask(img_path):
7     # Set model to evaluation mode (turns off Dropout)
8     model.eval()
9
10    # 1. Load the image
11    img = Image.open(img_path).convert('RGB')
12
13    # 2. Preprocess (Must match exactly what we did in Step 5)
14    test_transforms = transforms.Compose([
15        transforms.Resize((128, 128)),
16        transforms.ToTensor(),
17        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
18    ])
19
20    # Add batch dimension (.unsqueeze(0)) because the model expects [batch_size, C, H, W]
21    img_tensor = test_transforms(img).unsqueeze(0).to(device)
22
23    # 3. Predict
24    with torch.no_grad():
25        output = model(img_tensor)
26        # Convert raw logits to probability percentages
27        probabilities = torch.nn.functional.softmax(outputs, dim=1)
28        # Get the index of the highest score
29        _, predicted = torch.max(output, 1)
30
31    # 4. Map the number back to a human-readable label
32    classes = {0: "Without Mask", 1: "With Mask"}
33    prediction_text = classes[predicted.item()]
34    confidence = probabilities[0][predicted.item()].item() * 100
35    prob_without = probabilities[0][0].item()
36    prob_with = probabilities[0][1].item()
37
38    # FIX: Print them matching their true underlying index position
39    print(f"Confidences -> Without Mask (Index 0): {prob_without:.4f} | With Mask (Index 1): {prob_with:.4f}")
40

```

```
41 # 5. Visualize
42 plt.imshow(img)
43 plt.title(f"Result: {prediction_text}")
44 plt.axis('off')
45 plt.show()
46
47 # Replace 'my_test_image.jpg' with the actual name of your file
48 predict_mask('without_mask_men2.jpg')
```

Confidences -> Without Mask (Index 0): 0.0000 | With Mask (Index 1): 1.0000

Result: Without Mask



```
1 # Save the model weights to a file
2 torch.save(model.state_dict(), 'my_custom_model.pth')
3
4 # Download the file to your computer so you don't lose it when Colab resets
5 from google.colab import files
6 files.download('my_custom_model.pth')
```

```
1 # Mount Drive
2 from google.colab import drive
3 drive.mount('/content/drive')
4
5 # Save model permanently to your Drive
6 import shutil
7 shutil.copy('my_custom_model.pth', '/content/drive/MyDrive/my_custom_model.pth')
8 print("Model saved to Google Drive!")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
-----  
FileNotFoundError                               Traceback (most recent call last)  
/tmp/ipykernel_1428/1292397016.py in <cell line: 0>()  
      5 # Save model permanently to your Drive  
      6 import shutil  
----> 7 shutil.copy('my_custom_model.pth', '/content/drive/MyDrive/my_custom_model.pth')  
      8 print("Model saved to Google Drive!")
```

```
----- 1 frames -----  
/usr/lib/python3.12/shutil.py in copyfile(src, dst, follow_symlinks)  
    258     os.symlink(os.readlink(src), dst)  
    259     else:  
--> 260         with open(src, 'rb') as fsrc:  
    261             try:  
    262                 with open(dst, 'wb') as fdst:
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'my_custom_model.pth'
```

✓ Step 11: Generating the Confusion Matrix

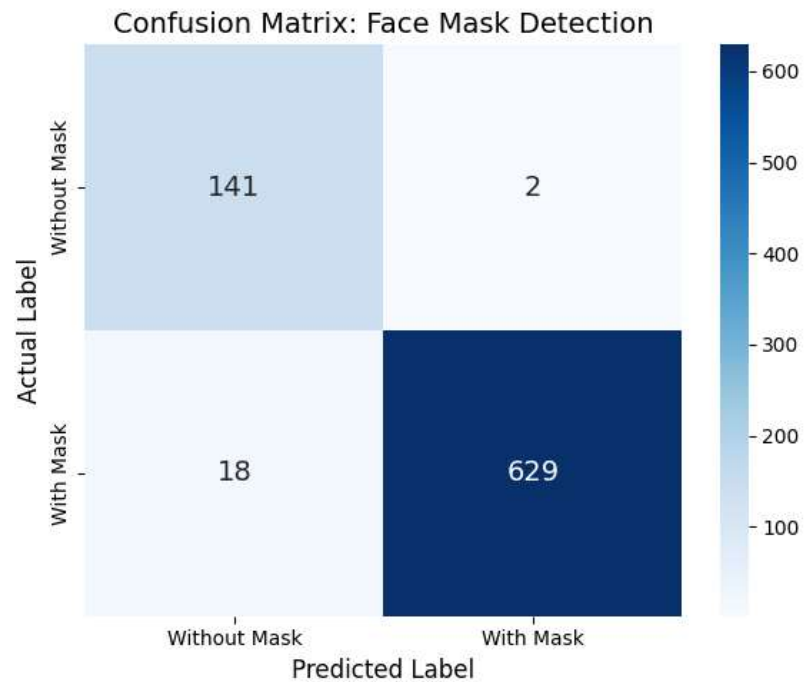
```
1 # To do this in PyTorch, we need to collect all the "True Labels" and "Predicted Labels" from your val_loader.
```

```
1 from sklearn.metrics import confusion_matrix  
2 import seaborn as sns  
3 import pandas as pd  
4 import matplotlib.pyplot as plt  
5 import torch  
6  
7 # Store predictions and true labels  
8 all_preds = []  
9 all_labels = []  
10  
11 # Evaluation mode  
12 model.eval()  
13  
14 with torch.no_grad():  
15     for images, labels in val_loader:  
16         images = images.to(device)  
17         # Note: You don't strictly need to move labels to GPU for evaluation,  
18         # but keeping it is completely fine!  
19         labels = labels.to(device)  
20  
21         outputs = model(images)  
22  
23         # Get predicted class  
24         _, preds = torch.max(outputs, 1)  
25  
26         # Move to CPU before converting to numpy  
27         all_preds.extend(preds.cpu().numpy())  
28         all_labels.extend(labels.cpu().numpy())  
29  
30 # Create confusion matrix (Will automatically be a 2x2 matrix now)
```

```

31 cm = confusion_matrix(all_labels, all_preds)
32
33 # FIX: Class names dropped down to 2 items to match our new configuration
34 classes = ["Without Mask", "With Mask"]
35
36 # Convert to DataFrame (Perfect 2x2 mapping)
37 cm_df = pd.DataFrame(cm, index=classes, columns=classes)
38
39 # Plot heatmap
40 plt.figure(figsize=(6, 5)) # Slightly smaller size fits a 2x2 matrix better
41 sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', annot_kws={"size": 14})
42
43 plt.title('Confusion Matrix: Face Mask Detection', fontsize=14)
44 plt.ylabel('Actual Label', fontsize=12)
45 plt.xlabel('Predicted Label', fontsize=12)
46
47 plt.tight_layout()
48 plt.show()

```



Transfer Learning" with MobileNetV2

It's the industry standard that would likely fix the "glasses" error entirely.

```

1 # Why MobileNetV2?
2 # MobileNetV2 is incredibly fast and lightweight. It was designed by Google to run on mobile phones.
3 # Most importantly, it has already seen thousands of people wearing glasses, so it knows that glasses are part of a face, not a mask

```

Step 12: Implementing Transfer Learning

```
1 # from torchvision import models
2
3 # # 1. Load the pre-trained MobileNetV2
4 # # 'weights=models.MobileNet_V2_Weights.DEFAULT' downloads the brain trained on ImageNet
5 # model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
6
7 # # 2. "Freeze" the base layers
8 # # We don't want to change the part of the brain that already knows how to see edges/colors.
9 # for param in model.parameters():
10 #     param.requires_grad = False
11
12 # # 3. Replace the final layer (The Classifier)
13 # # MobileNetV2 originally has 1000 outputs. We only need 3.
14 # num_features = model.classifier[1].in_features
15 # model.classifier[1] = nn.Sequential(
16 #     nn.Dropout(0.2),
17 #     nn.Linear(num_features, 3) # Our 3 classes
18 # )
19
20 # # 4. Move to GPU
21 # model = model.to(device)
22
23 # print("Pre-trained MobileNetV2 is ready!")
```

Downloading: "https://download.pytorch.org/models/mobilenet_v2-7ebf99e0.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v2-7ebf99e0.pth
100%|██████████| 13.6M/13.6M [00:00<00:00, 100MB/s]
Pre-trained MobileNetV2 is ready!

1 Start coding or generate with AI.

```
1 # import cv2
2 # from PIL import Image
3 # import torch
4 # import matplotlib.pyplot as plt
5 # from torchvision import transforms
6
7 # def predict_automated(img_path):
8 #     # 1. Load the full image
9 #     image_cv = cv2.imread(img_path)
10 #     if image_cv is None:
11 #         print("Error: Could not find image. Check the path!")
12 #         return
13
14 #     # 2. Convert to Grayscale for the detector to work
15 #     gray = cv2.cvtColor(image_cv, cv2.COLOR_BGR2GRAY)
16
17 #     # 3. Load the pre-trained Face Detector
18 #     face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
19
20 #     # 4. Find all faces in the photo
21 #     faces_detected = face_cascade.detectMultiScale(gray, 1.1, 4)
22
23 #     if len(faces_detected) == 0:
```

```
24 #         print("No face detected! Try a clearer photo.")
25 #         return
26
27 #     print(f"Found {len(faces_detected)} face(s). Processing...")
28
29 #     # --- DEFINING TRANSFORMS HERE TO FIX NAMEERROR ---
30 #     local_transforms = transforms.Compose([
31 #         transforms.Resize((128, 128)),
32 #         transforms.ToTensor(),
33 #         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
34 #     ])
35
36 #     for (x, y, w, h) in faces_detected:
37 #         # 5. CROP the face
38 #         face_crop = image_cv[y:y+h, x:x+w]
39 #         face_rgb = cv2.cvtColor(face_crop, cv2.COLOR_BGR2RGB)
40 #         face_pil = Image.fromarray(face_rgb)
41
42 #         # 6. Predict using the MobileNetV2 model
43 #         img_tensor = local_transforms(face_pil).unsqueeze(0).to(device)
44
45 #         model.eval()
46 #         with torch.no_grad():
47 #             output = model(img_tensor)
48 #             _, predicted = torch.max(output, 1)
49
50 #         classes = {0: "Without Mask", 1: "With Mask", 2: "Mask Worn Incorrectly"}
51 #         result = classes[predicted.item()]
52
53 #         # 7. Visualize
54 #         plt.imshow(face_rgb)
55 #         plt.title(f"Prediction: {result}")
56 #         plt.axis('off')
57 #         plt.show()
58
59 # # RUN IT!
60 # predict_automated('incorrect_mask_wear.png')
```

Found 1 face(s). Processing...

Prediction: Without Mask

